Student Registration System

Combined CI/CD Pipeline Submission

Flask • MongoDB • Jenkins • GitHub Actions • Pytest

Project report and implementation guide

Project	Student Registration System (Flask CRUD application)
Database	MongoDB via Flask-PyMongo
CI/CD tools	Jenkins Pipeline and GitHub Actions
Deployment flow	Test and package on pushes; staging deploy on staging branch; production deploy on version tag
Submission fields	GitHub repository URL and Jenkins/GitHub Actions screenshots to be added after execution

Prepared as a combined submission template. Run the pipelines, add your repository URL, and insert real execution screenshots before final submission.

1. Project Overview

This submission combines the two CI/CD assignments into one delivery plan for a Flask-based Student Registration System. The application supports creating, listing, editing, and deleting student records stored in MongoDB. Jenkins and GitHub Actions both automate dependency installation, tests, packaging, and deployment workflows.

Application features

• List all students on the home page.

• Add a student record.

• Update existing student details.

• Delete a student with a confirmation step.

• Responsive user interface using Bootstrap 5 and Jinja2 templates.

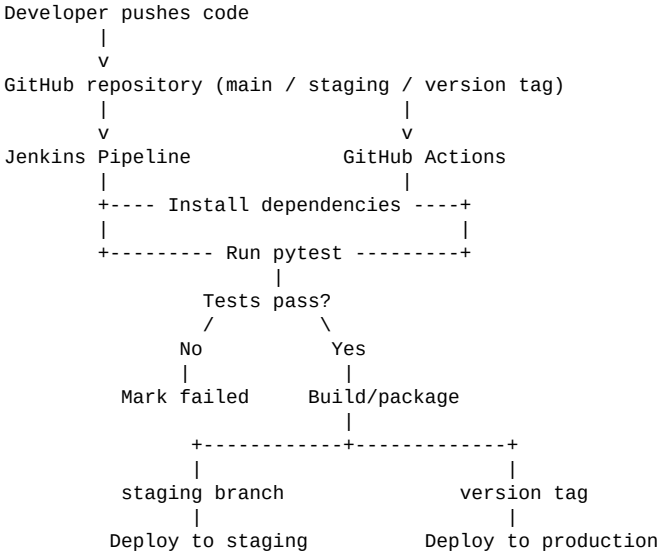
• Configuration through environment variables in a local .env file.

Technology stack

Layer	Technology
Backend	Python 3, Flask
Database	MongoDB, Flask-PyMongo
Frontend	HTML, Jinja2, Bootstrap 5
Testing	pytest
CI/CD	Jenkins Pipeline and GitHub Actions
Configuration	Environment variables / GitHub Secrets

2. Combined CI/CD Design

A single GitHub repository is the source of truth. The Jenkins pipeline and GitHub Actions workflow can run independently against the same code. Both should use the same test suite so that a change is validated consistently.



Important: configure only the deployment target that you actually have. The sample pipelines below use an SSH-based Linux staging/production host. If your application is deployed to a different platform, replace the deployment commands with the appropriate provider-specific steps.

3. Repository Structure

```
student-registration-system/
■■■ app.py
■■■ requirements.txt
■■■ pytest.ini
■■■ tests/
■   ■■■ test_app.py
■■■ templates/
■   ■■■ base.html
■   ■■■ index.html
■   ■■■ add_student.html
■   ■■■ update_student.html
■■■ .github/
■   ■■■ workflows/
■       ■■■ flask-ci-cd.yml
■■■ Jenkinsfile
■■■ README.md
■■■ .gitignore
```

Do not commit .env, passwords, private keys, MongoDB credentials, or production secrets. Add .env and virtual-environment folders to .gitignore.

4. Application Dependencies and Tests

requirements.txt

```
Flask
Flask-PyMongo
python-dotenv
pytest
certifi
```

The bson package is normally provided by PyMongo. Avoid installing the separate package named bson; install Flask-PyMongo, which brings PyMongo and its compatible bson module.

Suggested pytest smoke tests

Create tests/test_app.py. These example tests assume app.py exposes app and that routes use standard Flask endpoints. Adapt endpoint names and MongoDB mocking to match the actual application implementation.

```
import os
import pytest
from app import app as flask_app

@pytest.fixture()
def client():
    flask_app.config.update(TESTING=True)
    with flask_app.test_client() as test_client:
        yield test_client

def test_home_page_responds(client):
    response = client.get("/")
    assert response.status_code == 200

def test_add_student_page_responds(client):
    # Change the URL if your application uses a different route.
    response = client.get("/add")
    assert response.status_code in (200, 302, 404)
```

For stronger tests, mock the MongoDB collection and assert that insert, update, and delete operations are called correctly. The smoke tests above do not verify CRUD database behavior.

5. Jenkins CI/CD Pipeline

Prerequisites

• Jenkins installed on a VM or server with Git and a supported Python 3 version available to the Jenkins agent.

- Jenkins Pipeline and Git plugins installed.
- A GitHub repository containing the application, Jenkinsfile, requirements.txt, and tests.
- A staging Linux host reachable from Jenkins, if deploying via SSH.
- Jenkins credentials configured for deployment and email SMTP.

Jenkinsfile

```
pipeline {
    agent any

    options {
        timestamps()
        disableConcurrentBuilds()
    }

    environment {
        VENV = ".venv"
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build - Install Dependencies') {
            steps {
                sh '''
                    python3 --version
                    python3 -m venv "$VENV"
                    . "$VENV/bin/activate"
                    python -m pip install --upgrade pip
                    pip install -r requirements.txt
                '''
            }
        }

        stage('Test - pytest') {
            steps {
                sh '''
                    . "$VENV/bin/activate"
                    pytest -q
                '''
            }
        }

        stage('Build - Package') {
            steps {
                sh '''
                    tar --exclude=.git --exclude=.venv --exclude=.env \
                        -czf flask-app-${BUILD_NUMBER}.tar.gz .
                '''
                archiveArtifacts artifacts: 'flask-app-*.tar.gz',
                                 fingerprint: true
            }
        }

        stage('Deploy - Staging') {
            when {
                branch 'staging'
            }
            steps {
                echo 'Add your approved staging deployment command here.'
                // Example: use sshagent with a Jenkins SSH credential
                // and copy the artifact to the staging server.
            }
        }
    }

    post {
        success {
            echo 'CI/CD pipeline succeeded.'
        }
        failure {
            echo 'CI/CD pipeline failed.'
```

```
        }
        always {
            cleanWs()
        }
    }
}
```

The Jenkinsfile intentionally leaves deployment as a clearly marked configuration point rather than pretending a staging server is already known. For a real deployment, add the SSH credential ID and host/path for your environment. Jenkins email notifications can be configured with the Email Extension plugin and SMTP settings; use emailext in post { success/failure } after configuring it.

Jenkins trigger and email notification

• Create a Pipeline job and select “Pipeline script from SCM”; point it to the repository and Jenkinsfile.

• For multibranch builds, configure the GitHub Branch Source plugin and GitHub webhook to trigger branch indexing/builds.

• In GitHub repository settings, add a webhook pointing to the Jenkins webhook endpoint if required by the chosen Jenkins integration.

• Configure SMTP under Jenkins system settings and install/configure the Email Extension plugin.

• Add success and failure email recipients using emailext in the post block. Do not place SMTP passwords in the Jenkinsfile.

6. GitHub Actions CI/CD Workflow

Save the following as .github/workflows/flask-ci-cd.yml. It tests pushes and pull requests, deploys to staging when the staging branch is updated, and deploys to production when a version tag such as v1.0.0 is pushed. Deployment jobs are examples and require matching GitHub Secrets.

```
name: Flask CI/CD

on:
  push:
    branches: [main, staging]
    tags: ['v*']
  pull_request:
    branches: [main, staging]

permissions:
  contents: read

jobs:
  test-build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: pip

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest

      - name: Run tests
        run: pytest -q

      - name: Build package
        run: |
          tar --exclude=.git --exclude=.venv --exclude=.env \
            -czf flask-app.tar.gz .

      - name: Upload artifact
```

```
      uses: actions/upload-artifact@v4
      with:
        name: flask-app
        path: flask-app.tar.gz

  deploy-staging:
    needs: test-build
    if: github.event_name == 'push' && github.ref == 'refs/heads/staging'
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - name: Download package
        uses: actions/download-artifact@v4
        with:
          name: flask-app

      - name: Configure SSH
        env:
          DEPLOY_KEY: ${{ secrets.STAGING_SSH_KEY }}
          DEPLOY_HOST: ${{ secrets.STAGING_HOST }}
          DEPLOY_USER: ${{ secrets.STAGING_USER }}
        run: |
          mkdir -p ~/.ssh
          printf '%s\n' "$DEPLOY_KEY" > ~/.ssh/id_ed25519
          chmod 600 ~/.ssh/id_ed25519
          ssh-keyscan -H "$DEPLOY_HOST" >> ~/.ssh/known_hosts
          scp flask-app.tar.gz "$DEPLOY_USER@$DEPLOY_HOST:/tmp/flask-app.tar.gz"
          ssh "$DEPLOY_USER@$DEPLOY_HOST"             'mkdir -p ~/apps/student-registration &&             tar -xzf /tm

  deploy-production:
    needs: test-build
    if: github.event_name == 'push' && startsWith(github.ref, 'refs/tags/v')
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Download package
        uses: actions/download-artifact@v4
        with:
          name: flask-app

      - name: Configure SSH
        env:
          DEPLOY_KEY: ${{ secrets.PRODUCTION_SSH_KEY }}
          DEPLOY_HOST: ${{ secrets.PRODUCTION_HOST }}
          DEPLOY_USER: ${{ secrets.PRODUCTION_USER }}
        run: |
          mkdir -p ~/.ssh
          printf '%s\n' "$DEPLOY_KEY" > ~/.ssh/id_ed25519
          chmod 600 ~/.ssh/id_ed25519
          ssh-keyscan -H "$DEPLOY_HOST" >> ~/.ssh/known_hosts
          scp flask-app.tar.gz "$DEPLOY_USER@$DEPLOY_HOST:/tmp/flask-app.tar.gz"
          ssh "$DEPLOY_USER@$DEPLOY_HOST"             'mkdir -p ~/apps/student-registration &&             tar -xzf /tm
```

Security note: ssh-keyscan trusts the key presented at connection time and is not a substitute for verifying the host fingerprint out-of-band. For a stricter setup, pin the known_hosts entry in a GitHub Secret or repository variable. The example transfers and extracts files only; it does not restart a service. Add a systemd, container, or platform-specific release step after configuring the target host.

7. Required GitHub Secrets and Environments

Secret	Purpose
STAGING_SSH_KEY	Private SSH key for staging deployment user
STAGING_HOST	Staging server DNS name or IP
STAGING_USER	SSH username on staging server
PRODUCTION_SSH_KEY	Private SSH key for production deployment user
PRODUCTION_HOST	Production server DNS name or IP
PRODUCTION_USER	SSH username on production server

• In GitHub, open Settings → Secrets and variables → Actions and create the required secrets.

• Create staging and production environments under Settings → Environments.

• Protect production with required reviewers and restrict deployments to version tags where possible.

• Store MongoDB URI and Flask SECRET_KEY as runtime secrets on the deployment host or environment—not in source code or the artifact.

• Never expose secret values in logs or screenshots.

8. Branching and Release Strategy

• main: stable application code; CI tests run on push and pull request.

• staging: changes approved for staging; passing tests trigger staging deployment.

• Version tags (for example v1.0.0): release candidates for production; passing tests trigger production deployment.

• Recommended: protect main and staging with pull-request reviews and required passing checks.

9. README.md Documentation Template

```
# Student Registration System — CI/CD

## Overview
Flask CRUD application for student records using MongoDB, with automated
testing and packaging through Jenkins and GitHub Actions.

## Local setup
1. Install Python 3.12+ and make MongoDB available.
2. Create a virtual environment and activate it.
3. Install dependencies: `pip install -r requirements.txt`.
4. Create `.env` with `MONGO_URI` and `SECRET_KEY`.
5. Run `python app.py` and open the configured local URL.

## Tests
Run `pytest -q`.

## GitHub Actions
- Push/PR to `main` or `staging`: install dependencies, run tests, package.
- Push to `staging`: deploy package to staging after tests pass.
- Push a version tag such as `v1.0.0`: deploy package to production after tests pass.
- Configure `STAGING_SSH_KEY`, `STAGING_HOST`, `STAGING_USER`,
  `PRODUCTION_SSH_KEY`, `PRODUCTION_HOST`, and `PRODUCTION_USER` as Actions secrets.

## Jenkins
Create a Pipeline or Multibranch Pipeline job that loads `Jenkinsfile`
from SCM. Configure GitHub webhook/branch discovery, Python on the agent,
SMTP settings, and deployment credentials. Configure the deployment stage
for your actual staging host.

## Security
Never commit `.env`, private keys, database credentials, or deployment secrets.
Use protected GitHub environments for production deployments.

## Evidence for submission
- Add the GitHub repository URL.
- Add screenshots of a successful Jenkins run showing Build, Test, Deploy.
- Add screenshots of a successful GitHub Actions run showing tests/build,
  staging deployment, and production deployment (tagged release).
```

10. End-to-End Implementation Checklist

• Fork or create a GitHub repository from the supplied Flask project.

• Review app.py routes and update pytest tests to match the actual endpoints.

• Verify requirements.txt installs cleanly in a fresh Python virtual environment.

• Create main and staging branches.

• Commit Jenkinsfile, workflow YAML, tests, README.md, and .gitignore.

• Run pytest locally and fix failures.

• Configure Jenkins job, GitHub integration/webhook, SMTP, and deployment credentials.

• Configure GitHub Actions secrets and protected staging/production environments.

• Configure actual staging and production hosts and add service restart/health-check commands.

• Push a commit to main and staging; create a version tag such as v1.0.0.

• Verify each workflow and deployment succeeds; capture screenshots from real runs.

• Replace placeholders in the submission section below before submitting.

11. Submission Evidence

GitHub repository URL: _____

Jenkins job URL: _____

GitHub Actions run URL: _____

Staging URL (if available): _____

Insert genuine screenshots here in the final submitted version:

• Jenkins pipeline view showing Checkout/Build, Test, and Deploy stages.

• Jenkins build console or test report showing pytest success.

• GitHub Actions run showing successful test/build job.

• GitHub Actions staging deployment run from the staging branch.

• GitHub Actions production deployment run triggered by a version tag.

Screenshots and repository URLs are intentionally left as fields to complete after running the implementation. This document is a submission-ready guide/template, not proof that the pipelines have already executed.

12. References

Flask documentation: https://flask.palletsprojects.com/

Flask-PyMongo documentation: https://flask-pymongo.readthedocs.io/

pytest documentation: https://docs.pytest.org/

Jenkins Pipeline documentation: https://www.jenkins.io/doc/book/pipeline/

GitHub Actions documentation: https://docs.github.com/actions

Original sample repository supplied in the assignment: https://github.com/mohanDevOps-arch/flask_Practice